
ECM1414 COURSEWORK PRESENTATION

(Group 14)

PROBLEM INTRODUCTION

- Event planner
- Given a set of distinct events with parameters of:
 - Name
 - Time required
 - Cost
 - Enjoyment Value

Goal: Given restrictions on budget and time available, find a combination of events which will maximize the enjoyment value



```

for l in range(1, number_of_events + 1)

    for subset in itertools.combinations(event_list, l)

        if validset(subset) is True

            event_combo_list is event_combo_list + subset

        END if

    END for

END for

for i in range(0, event_combo_list_length)

    info is sum(event_combo_list[i])

    subsets_with_details.add(event_list, info)

END for

subsets_with_details is subsets_with_details + [event_combo_list[i],info]

best_combo_numeric is max of: subsets_with_details[enjoyment]

return best_combo, details

```

ALGORITHMS EXPLAINED: BRUTE-FORCE APPROACH

- Create a list of all possible solutions
 - Filter out invalid sets
 - Of the valid sets, select the one that maximizes enjoyment
 - It's simple but slow
-

ALGORITHMS EXPLAINED: DYNAMIC APPROACH

PART 1: MEMOIZATION:

- Breaks down the problem into many sub-problems
- Represents each sub-problem via a grid of all possible constraints up to the constraints of the given problem
- Uses values from previous sub-problems to calculate future ones

```
function Event_Planner_Dynamic:
    data_field_2d = [[0 for _ in range(max_budget + 1)] for _ in range(number of events + 1)]

    for i in range(n+1)

        for m in range(max_budget+1)

            if i is 0 or m is 0

                set data_field_2d[i][m] to 0
            END if
            else if event_list[i-1][1] is less than or equal to m:

                data_field_2d[i][m] = max of(
                    data_field_2d[i-1][m]
                    and
                    event_list[i-1][2] + data_field_2d[i-1][m-event_list[i-1][1]]
                )
            END else if
            else:

                set data_field_2d[i][m] to data_field_2d[i-1][m]

            END else

        END for

    END for
```

ALGORITHMS EXPLAINED: DYNAMIC APPROACH

PART 2: DEDUCTION:

- Once the maximum enjoyment has been calculated, the event list is derived from that value
- That's done through working backwards from the value of the current cell and information gathered from surrounding cells

```
i is number_of_events
```

```
j is max_budget
```

```
while x and y is greater than 0
```

```
    if data_field_2d[x][y] is not data_field_2d[x-1][y]
```

```
        solution is solution + [event_list[x-1]]
```

```
        j is j - event_list[x-1][1]
```

```
    END if
```

```
    decrement x by 1
```

```
END while
```

```
return best_combo, details
```

```
END function Event_Planner_Dynamic
```

ALGORITHMS EXPLAINED: DYNAMIC APPROACH

KEY DIFFERENCES AND ADVANTAGES

- Reduces redundant computing due to memoization
 - The change in approach changes the time complexity to a lower order (linear change instead of an exponential change in this case)
 - Which makes it compute faster for increasingly larger problems
-

DEMONSTRATION



RESULTS AND DISCUSSION: COMPARISON



RESULTS AND DISCUSSION: FINDINGS



QUESTIONS?

